

1 Реализация VPN-сервера на базе WireGuard

1.1 Обоснование выбора протокола WireGuard

Для построения защищённого VPN-канала был выбран протокол WireGuard. Данное решение было принято на основе анализа современных VPN-протоколов по следующим критериям:

- **Производительность** — WireGuard работает на уровне ядра Linux, что обеспечивает высокую скорость передачи данных и низкую задержку;
- **Безопасность** — используются современные криптографические алгоритмы: Curve25519 для обмена ключами, ChaCha20 для шифрования и Poly1305 для аутентификации;
- **Простота настройки** — конфигурационный файл содержит минимальное количество параметров, что снижает вероятность ошибок;
- **Кроссплатформенность** — официальные клиенты доступны для Windows, macOS, Linux, Android и iOS;
- **Надёжность** — протокол автоматически восстанавливает соединение при обрыве и поддерживает роуминг между сетями.

Таким образом, WireGuard был выбран как оптимальное решение для создания защищённого туннеля между клиентом и сервером в рамках данной работы.

1.2 Архитектура разработанного решения

Разработанное VPN-решение состоит из трёх основных компонентов:

1. **Серверная часть** — удалённый VPS-сервер (Ubuntu 22.04), на котором развёрнут WireGuard. Сервер выполняет функции маршрутизатора, перенаправляя трафик клиента в интернет и обратно.
2. **Клиентская часть** — приложение с графическим интерфейсом, разработанное на Python с использованием фреймворка PyQt5. Приложение обеспечивает управление подключением, отображение статуса и статистики.
3. **Транспортный уровень** — зашифрованный туннель, создаваемый протоколом WireGuard. Туннель работает поверх UDP и обеспечивает конфиденциальность и целостность передаваемых данных.

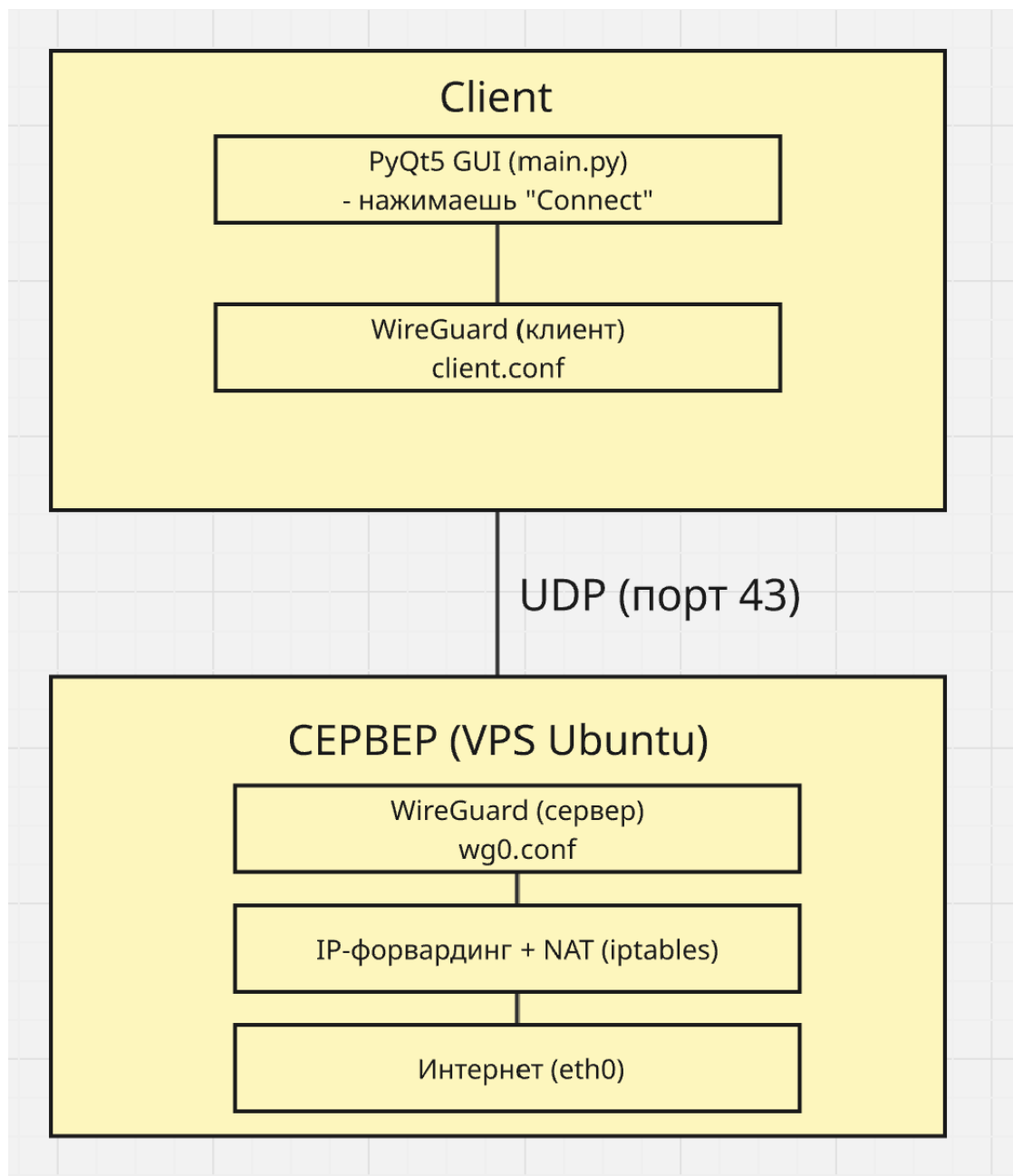


Рис. 1: Архитектура VPN-решения

На рисунке 1 представлена схема взаимодействия компонентов разработанного решения.

1.3 Подготовка серверной инфраструктуры

Для развёртывания VPN-сервера был использован VPS-сервер, предоставляемый хостинг-провайдером Fornex. Технические характеристики сервера:

- Операционная система: Ubuntu 22.04 LTS
- Публичный IP-адрес: 89.127.203.246
- Доступ: root по SSH

Перед установкой WireGuard была выполнена базовая настройка сервера: обновление пакетов и установка необходимых утилит.

```
1 sudo apt update
2 sudo apt upgrade -y
3 sudo apt install curl wget net-tools -y
```

Листинг 1: Базовая настройка сервера

1.4 Установка и настройка WireGuard на сервере

1.4.1 Установка пакетов

Установка WireGuard выполнялась из официальных репозиториях Ubuntu:

```
1 sudo apt install wireguard wireguard-tools -y
```

Листинг 2: Установка WireGuard

После установки была проверена версия установленного пакета:

```
1 wg --version
```

Листинг 3: Проверка версии WireGuard

1.4.2 Генерация криптографических ключей

Для обеспечения безопасности соединения были сгенерированы пары ключей для сервера и для клиента. Ключи генерировались с использованием встроенных утилит WireGuard:

```
1 #
2 mkdir -p /etc/wireguard/keys
3 cd /etc/wireguard/keys
4
5 #
6 wg genkey | tee server_private.key | wg pubkey > server_public.key
7
8 #
9 wg genkey | tee client_private.key | wg pubkey > client_public.key
10
11 #                                     (
12                                     root)
12 chmod 600 *.key
```

Листинг 4: Генерация ключей

В результате были получены следующие ключи:

- `server_private.key` — приватный ключ сервера (хранится в секрете);
- `server_public.key` — публичный ключ сервера (передаётся клиенту);
- `client_private.key` — приватный ключ клиента (передаётся клиенту);
- `client_public.key` — публичный ключ клиента (добавляется в конфиг сервера).

1.4.3 Создание конфигурационного файла сервера

Конфигурационный файл сервера был создан по пути `/etc/wireguard/wg0.conf`. Его содержимое:

```
1 [Interface]
2 Address = 10.66.66.1/24
3 ListenPort = 51820
4 PrivateKey = <server_private.key>
5
6 #
7 PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
8 PostUp = iptables -A FORWARD -o wg0 -j ACCEPT
9 PostUp = iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
10
11 #
12 PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
13 PostDown = iptables -D FORWARD -o wg0 -j ACCEPT
14 PostDown = iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE
15
16 [Peer]
17 PublicKey = <client_public.key>
18 AllowedIPs = 10.66.66.2/32
```

Листинг 5: Конфигурационный файл сервера `/etc/wireguard/wg0.conf`

Параметры конфигурации представлены в таблице 1.

Параметр	Значение	Назначение
Address	10.66.66.1/24	IP-адрес сервера в VPN-сети
ListenPort	51820	Порт, на котором сервер принимает подключения
PrivateKey	<ключ>	Приватный ключ сервера
PostUp	iptables ...	Команды, выполняемые при запуске интерфейса
PostDown	iptables ...	Команды, выполняемые при остановке интерфейса
PublicKey	<ключ клиента>	Публичный ключ клиента для аутентификации
AllowedIPs	10.66.66.2/32	IP-адрес, назначаемый клиенту

Таблица 1: Параметры конфигурации сервера

1.4.4 Настройка маршрутизации и NAT

Для того чтобы клиент мог выходить в интернет через сервер, была выполнена настройка IP-форвардинга и NAT.

Включение IP-форвардинга:

```
1 #
2 echo 1 > /proc/sys/net/ipv4/ip_forward
```

```

3
4 #
5 echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf
6 sysctl -p

```

Листинг 6: Включение IP-форвардинга

Настройка NAT (маскарадинг):

```

1 iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

```

Листинг 7: Настройка NAT

Данная команда подменяет исходный IP-адрес клиента на IP-адрес сервера, благодаря чему ответные пакеты от интернет-ресурсов приходят на сервер и перенаправляются клиенту.

1.4.5 Запуск сервера и настройка автозагрузки

Запуск WireGuard-интерфейса:

```

1 wg-quick up wg0

```

Листинг 8: Запуск WireGuard

Проверка статуса:

```

1 wg show

```

Листинг 9: Проверка статуса

Результат выполнения команды подтвердил успешный запуск интерфейса:

```

1 interface: wg0
2   public key: <                                --                                >
3   private key: (hidden)
4   listening port: 51820

```

Листинг 10: Вывод команды wg show

Настройка автоматического запуска при загрузке системы:

```

1 systemctl enable wg-quick@wg0

```

Листинг 11: Настройка автозагрузки

1.5 Разработка клиентского приложения на PyQt5

1.5.1 Структура приложения

Для управления VPN-подключением было разработано кроссплатформенное приложение с графическим интерфейсом на Python с использованием фреймворка PyQt5. Приложение имеет следующую структуру:

```

client/
  main.py           # Точка входа
  wg_controller.py  # Управление WireGuard
  config_manager.py # Работа с конфигурациями
  ui/

```

```
main_window.py      # Главное окно
settings_dialog.py  # Окно настроек
configs/
  client.conf        # Конфигурационный файл клиента
```

1.5.2 Реализованный функционал

В процессе разработки клиентского приложения были реализованы следующие функции:

- **Управление подключением:**
 - Запуск и остановка VPN-туннеля;
 - Отображение текущего статуса (подключено / отключено);
 - Индикация состояния (цветовой индикатор).
- **Работа с конфигурациями:**
 - Импорт конфигурационного файла;
 - Сохранение пути к конфигурации;
 - Проверка наличия WireGuard на системе.
- **Отображение статистики:**
 - Количество принятых и переданных данных;
 - Время активного соединения;
 - Индикация качества сигнала.
- **Ведение логов:**
 - Отображение событий в реальном времени;
 - Запись времени выполнения операций;
 - Информирование об ошибках.

1.5.3 Модуль управления WireGuard

Основной модуль `wg_controller.py` обеспечивает взаимодействие с WireGuard через системные вызовы. В нём реализованы следующие методы:

- `connect(config_path)` — запускает VPN-подключение;
- `disconnect()` — останавливает VPN-подключение;
- `is_connected()` — проверяет статус соединения;
- `get_status()` — получает статистику трафика.

Пример реализации метода `connect()`:

```

1 def connect(self, config_path):
2     try:
3         if self.os_type == "Windows":
4             result = subprocess.run(
5                 ['wg', 'setconf', 'wg0', config_path],
6                 capture_output=True, text=True
7             )
8             if result.returncode != 0:
9                 print(f"Error: {result.stderr}")
10                return False
11            subprocess.run(
12                ['netsh', 'interface', 'set', 'interface', 'wg0', '
13                 enabled'],
14                capture_output=True
15            )
16        else:
17            subprocess.run(
18                ['sudo', 'wg-quick', 'up', config_path],
19                check=True, capture_output=True
20            )
21        return True
22    except Exception as e:
23        print(f"Connection error: {e}")
24        return False

```

Листинг 12: Метод connect() модуля wg_controller.py

1.5.4 Интерфейс пользователя

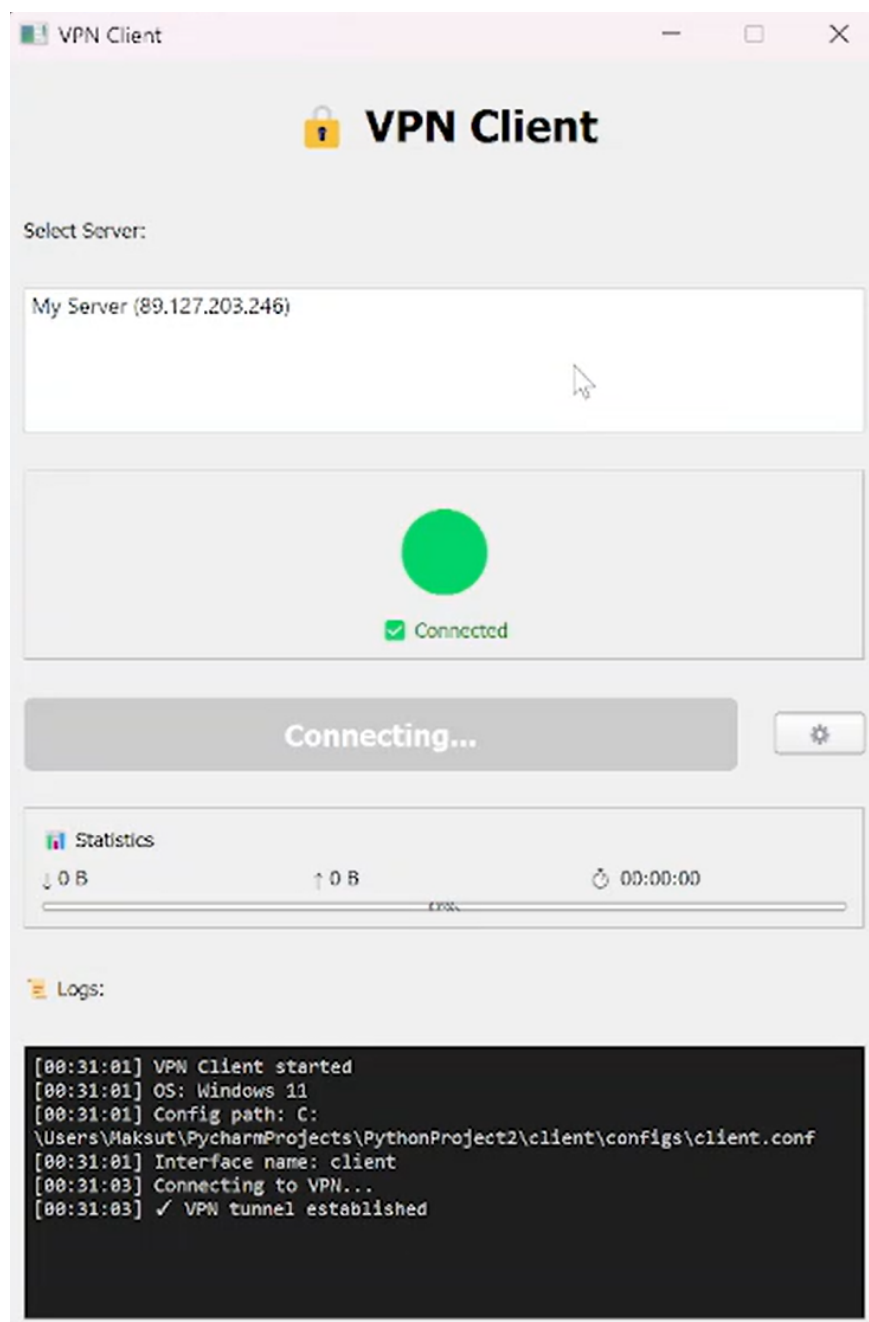


Рис. 2: Интерфейс клиентского приложения на PyQt5

Графический интерфейс приложения (рисунок 2) содержит следующие элементы:

- Заголовок с названием приложения;
- Список доступных серверов (для выбора);
- Индикатор статуса подключения (цветовой и текстовый);
- Кнопка управления подключением (Connect/Disconnect);
- Панель статистики трафика;
- Область вывода логов.

Стилизация интерфейса выполнена с использованием CSS-подобных стилей Qt, что обеспечивает современный внешний вид.

1.6 Настройка клиентской части на Windows

Для подключения к VPN-серверу с клиентской машины под управлением Windows были выполнены следующие шаги.

1.6.1 Установка WireGuard на Windows

Официальный клиент WireGuard был загружен с официального сайта и установлен на систему. После установки в системе появляется графическое приложение для управления подключениями.

1.6.2 Создание клиентского конфигурационного файла

На основе ключей, сгенерированных на сервере, был создан конфигурационный файл `client.conf`:

```
1 [Interface]
2 PrivateKey = <                                --                                >
3 Address = 10.66.66.2/24
4 DNS = 8.8.8.8
5
6 [Peer]
7 PublicKey = <                                --                                >
8 Endpoint = 89.127.203.246:51820
9 AllowedIPs = 0.0.0.0/0
10 PersistentKeepalive = 25
```

Листинг 13: Клиентский конфигурационный файл `client.conf`

Параметры клиентского конфига представлены в таблице 2.

Параметр	Значение	Назначение
PrivateKey	<ключ>	Приватный ключ клиента
Address	10.66.66.2/24	IP-адрес клиента в VPN-сети
DNS	8.8.8.8	DNS-сервер для разрешения имён
PublicKey	<ключ сервера>	Публичный ключ сервера
Endpoint	89.127.203.246:51820	Адрес и порт сервера
AllowedIPs	0.0.0.0/0	Весь трафик направлять через VPN
PersistentKeepalive	25	Интервал отправки keepalive-пакетов

Таблица 2: Параметры конфигурации клиента

1.6.3 Импорт конфигурации в WireGuard GUI

Конфигурационный файл был импортирован через интерфейс WireGuard: кнопка "Add Tunnel" → "Import tunnel(s) from file". После импорта туннель появился в списке доступных подключений.

1.6.4 Активация подключения

После нажатия кнопки "Activate" было установлено защищённое соединение с сервером. Успешность подключения подтверждена командой:

```
1 wg show
```

Листинг 14: Проверка статуса подключения

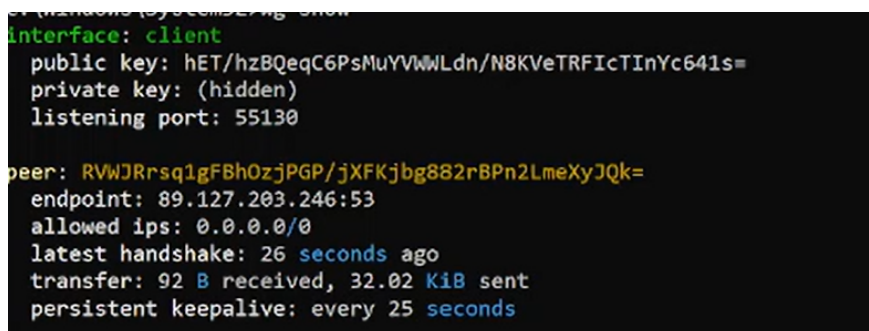
Результат выполнения показал наличие рукопожатия (latest handshake) и передачу данных:

```
1 interface: client
2   public key: <                                --                                >
3   private key: (hidden)
4   listening port: 56112
5
6 peer: <                                --                                >
7   endpoint: 89.127.203.246:51820
8   allowed ips: 0.0.0.0/0
9   latest handshake: 14 seconds ago
10  transfer: 92 B received, 14.52 KiB sent
11  persistent keepalive: every 25 seconds
```

Листинг 15: Вывод команды wg show на клиенте

1.7 Тестирование и проверка работоспособности

1.7.1 Проверка установления соединения

A screenshot of a terminal window showing the output of the 'wg show' command. The output is color-coded: 'interface: client' is green, 'public key' is blue, 'private key' is red, 'listening port' is green, 'peer:' is blue, 'endpoint' is green, 'allowed ips' is red, 'latest handshake' is blue, 'transfer' is green, and 'persistent keepalive' is blue. The values are: public key: hET/hzBQeqC6PsMuYVWwLdn/N8KVeTRFIcTinYc641s=, private key: (hidden), listening port: 55130, peer: RVWJRrsq1gFBh0zjPGP/jXFKjbg882rBPn2LmeXyJQk=, endpoint: 89.127.203.246:53, allowed ips: 0.0.0.0/0, latest handshake: 26 seconds ago, transfer: 92 B received, 32.02 KiB sent, persistent keepalive: every 25 seconds.

```
interface: client
public key: hET/hzBQeqC6PsMuYVWwLdn/N8KVeTRFIcTinYc641s=
private key: (hidden)
listening port: 55130

peer: RVWJRrsq1gFBh0zjPGP/jXFKjbg882rBPn2LmeXyJQk=
endpoint: 89.127.203.246:53
allowed ips: 0.0.0.0/0
latest handshake: 26 seconds ago
transfer: 92 B received, 32.02 KiB sent
persistent keepalive: every 25 seconds
```

Рис. 3: Результат выполнения команды wg show на клиенте

Для проверки соединения была выполнена команда на клиентской машине. Наличие строки latest handshake подтверждает успешное установление защищённого соединения между клиентом и сервером (рисунок 3).

1.7.2 Проверка маршрутизации трафика

Для проверки того, что весь интернет-трафик направляется через VPN-туннель, был выполнен запрос на определение внешнего IP-адреса:

```
1 curl ifconfig.me
```

Листинг 16: Проверка внешнего IP-адреса

Результат: 89.127.203.246 (IP-адрес сервера).

Это подтверждает, что трафик клиента успешно маршрутизируется через VPN-сервер

1.7.3 Проверка доступности интернета

Дополнительно была выполнена проверка доступности внешних ресурсов:

```
1 ping 8.8.8.8
2 nslookup google.com
```

Листинг 17: Проверка доступности интернета

Обе команды выполнились успешно, что подтверждает полную работоспособность VPN-соединения.

1.8 Выявленные проблемы и их решение

В процессе настройки и тестирования были выявлены следующие проблемы:

1.8.1 Проблема 1: Отсутствие интернета после подключения

Причина: Маршрут по умолчанию не был добавлен в таблицу маршрутизации Windows.

Решение: Добавление маршрута вручную:

```
1 route add 0.0.0.0 mask 0.0.0.0 10.66.66.1 metric 5
```

Листинг 18: Добавление маршрута в Windows

1.8.2 Проблема 2: Ошибка импорта конфигурации (illegal base64)

Причина: В публичный ключ сервера при копировании попал недопустимый символ (попросил нейронку считать ключ со скрина).

Решение: Тщательная проверка ключа и замена символов на корректные. Ключ должен содержать только символы base64: A-Z, a-z, 0-9, +, / и = в конце.

1.8.3 Проблема 3: Несовпадение публичных ключей

Причина: На сервере и клиенте использовались разные ключи.

Решение: Генерация новой пары ключей и синхронизация конфигурационных файлов.

1.9 Выводы по разделу

В рамках выполнения работы был успешно развёрнут и настроен VPN-сервер на базе протокола WireGuard. Выполнены следующие задачи:

- Установка и настройка WireGuard на сервере Ubuntu;
- Генерация криптографических ключей для сервера и клиента;
- Настройка IP-форвардинга и NAT для маршрутизации трафика;
- Создание конфигурационных файлов для сервера и клиента;

- Разработка клиентского приложения с графическим интерфейсом на PyQt5;
- Тестирование работоспособности VPN-соединения;
- Устранение выявленных проблем.

Разработанное решение обеспечивает:

- шифрование трафика с использованием современных криптографических алгоритмов;
- простоту управления через графический интерфейс;
- возможность использования на различных операционных системах;
- масштабируемость для добавления новых клиентов;
- автоматический запуск сервера при загрузке системы.